

AWS DeepRacer

Bilawal Singh Deu, Tanmay Nanda, Yash Sangtani, Zoya Ghoshal

Plaksha University, CSAI

[GitHub](#)

I. PROBLEM STATEMENT

Reinforcement Learning (RL) methods in the field of autonomous driving has been explored extensively in theory. However, solutions which typically adapt well to virtual environments, do not necessarily display the same results when it comes to the environments which simulate the real world. RL agents are required to make real-time decisions, which can often be difficult considering the dynamic and uncertain environments it encounters. Consider this, we decided to use AWS DeepRacer as a platform. Our project explores the applications of RL to train an autonomous racing car capable of navigating complex tracks or circuits. The key challenges include designing an effective state space representation, defining a robust action space, and finally, engineering a reward function that balances several things—track completion, speed optimization, and precision. We aim to develop RL models to improve performance when it comes to metrics like lap times, training efficiency, and generalization across many diverse tracks.

II. REINFORCEMENT LEARNING FORMULATION

In this project, the RL agent—which in this case is the AWS DeepRacer virtual race car model—has the primary objective of maximising its rewards while interacting with a dynamic environment. The agent tries to learn its optimal policy by exploring several actions, ultimately learning a policy which ends up giving it maximum rewards. The agent receives positive rewards if it stays on the track and receives negative rewards for going off-track. An episode in this case starts when the agent is somewhere on the specified track and ends either when the agent goes off-track or it completes a lap.

We tracked the progress of our agent through a simulation to understand things like:

- The agent's progress, where it goes off-track, and how the rewards increase or decrease through the agent's progression through the track.
- How changing certain hyperparameters like the steering angles, speeds, and tracks, among others, influences how the agent behaves.

States space

There are certain parameters which influence the state space of our virtual model. Some of them are defined as:

- Position coordinates: The (x, y) coordinates of the car on the track in the simulation.
- Orientation: The heading angle of the car relative to the track.
- Speed: The current velocity of the vehicle.
- Track parameters: Distance from centreline, track width, and wheels-on-track status, among others.
- Progress: Percentage of track completion.
- Waypoints: Coordinates marking the specific track or racing line.

Within the AWS DeepRacer simulation, we have the choice between a discrete or a continuous action space. A discrete action space represents all of the agent's possible actions for each state in the form of a finite set. For this project, it means that for every incrementally different environmental situation, the agent selects a speed and direction for the car (action) based on input from the environment. In a discrete action space, when approaching a turn, the car can choose to accelerate, brake, turn left, right, or go straight. These actions are defined as a combination of steering angle and speed creating a menu of options, 0-9, for the agent. A continuous action space allows the agent to instead choose an action from a range of values (once again, in terms of steering angle and speed). We are allowed to define the range of options the agent can choose from. Typically, discrete action spaces require a lower training time but lack adaptability due to the limitations on the actions that the agent can take. Continuous action spaces fill this gap but require a longer training time to choose the most optimal actions in accordance with its environment.

We were given the freedom to write our own reward function from scratch. A few baseline reward functions were given:

- Follow the centreline: This function sectioned the track into three specific zones. The closer the car was to the centre of the track, the higher its reward was.
- Stay within borders: This function defined the borders of the track and only rewarded the agent if all four wheels stayed within these borders. The onus was on the agent to figure out the most optimal path.
- Prevent zigzag: This function was essentially the same as the ‘follow the centreline’ reward function, except it had an additional restriction—the steering angle. This ensured the car drove smoothly down the middle.

While there were three race types—namely, time trial, object avoidance, and head-to-head racing— we focused on the time trial race type, which essentially gave us the objective of developing reward functions and hyperparameters to ensure the car completed the track in the least amount of time, while also steering smoothly. We were given a choice between two different training algorithms—Proximal Policy Optimisation (PPO) and Soft Actor Critic (SAC) — to allow the model to learn the policy and value functions simultaneously. PPO uses on-policy learning to gain an understanding of its environment through exploration and iterating on the current policy. The algorithm is stable but data hungry, which means it needs more information to perform better. However, it does encourage exploration by making the agent choose actions with higher entropy—to avoid convergence to a sub-optimal local maximum— thus allowing the agent to learn more about the environment and make informed decisions. SAC uses an off-policy learning strategy which means it uses observations used by previous policies’ exploration of the environment in order to iterate upon the current policy. This algorithm discards policies if they aren’t efficient enough, effectively making it less data hungry than PPO, but also more unstable. SAC is able to strike a good balance between exploration and exploitation, making it a good algorithm for training in the context of this project.

Some of the evaluation metrics used to gauge how well the model was performing were:

- Average return: The cumulative reward per episode.
- Completion rate: The percentage of successful track completions.
- Time to complete: Average time to finish the track.
- Track position accuracy: Deviation of the car from the optimal racing line.
- Stability: Consistency across multiple evaluation runs (in terms of lap times, trajectories, and rewards).
- Off-Track count: The number of times the car goes off-track in a lap.

We were also given the choice between two loss functions—Huber Loss and Mean Squared Error Loss—to understand how well the agent was learning in its environment. These objective functions were used to update the model’s weights, ensuring that the agent makes incremental changes to its strategy so that it gradually transforms from taking random actions to taking the most strategic actions for increasing the cumulative rewards. If the changes are too big, however, the training becomes unstable and ends up not learning. While both the loss functions behave similarly in terms of smaller updates, Huber loss tends to take smaller increments when it comes to larger updates, as compared to Mean Squared Error Loss. The choice of the loss function could vary according to what we wanted. If we wanted faster convergence, Huber Loss was a good choice. If faster training was required, Mean Squared Error Loss was the better option.

There were several other hyperparameters that could be tuned to make the model train and adapt better to its environment. Some of them were the gradient descent batch size, number of epochs during training, learning rate, entropy, discount factor, and finally, the number of experience episodes between each policy-updating iteration.

III. METHODOLOGY

A. PPO vs SAC -

PPO	SAC
Works in both discrete and continuous action spaces	Works in a continuous action space
On-Policy	Off-Policy
Uses entropy regularisation	Adds entropy to the maximisation objective

Table 1: Algorithm Comparison

a. PPO (Proximal Policy Optimization)

Proximal Policy Optimization (PPO) utilizes two neural networks: the policy network and the value network. The policy network, also known as the actor, selects actions based on the current state input. The value network, or critic, estimates the expected cumulative reward from a given state.

The policy network directly interacts with the environment or simulator and is responsible for controlling the agent—in this case, the Car. PPO employs on-policy learning, meaning it updates the value function using data collected from the current version of the policy during exploration. While on-policy methods often require more data compared to off-policy algorithms, they tend to offer improved training stability. As training progresses, the policy becomes increasingly deterministic. Weight updates favour actions that have led to higher rewards in the past, thus encouraging exploitation of learned behaviour while gradually reducing exploration.

b. SAC (Soft Actor Critic)

Soft Actor-Critic (SAC) utilizes off-policy learning, which separates the behavior policy from the target policy (also known as the objective policy). The behavior policy is used to interact with the environment and gather experience, while the target policy is updated using this collected data. This allows the target policy to learn not just from its own experiences, but also from data generated by previous versions of the policy.

Off-policy methods like SAC are typically more data-efficient, as they can reuse past experiences multiple times during training. However, this comes at the cost of reduced stability compared to on-policy algorithms. SAC addresses some of this instability through techniques like entropy regularization, which encourages exploration and leads to more robust learning.

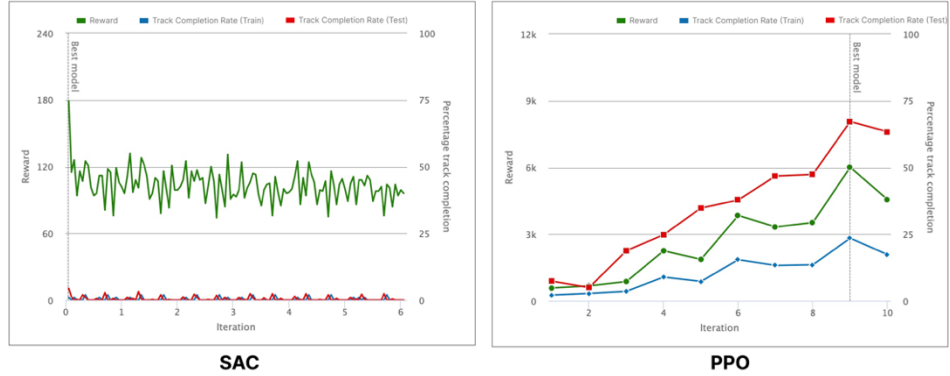


Fig 1: PPO vs SAC Policy Comparison

In our experience, PPO was much more stable and reliable as compared to SAC. As seen in the above figure, when trained on the same track with the same reward function and parameters, there is a drastic difference between the two algorithms.

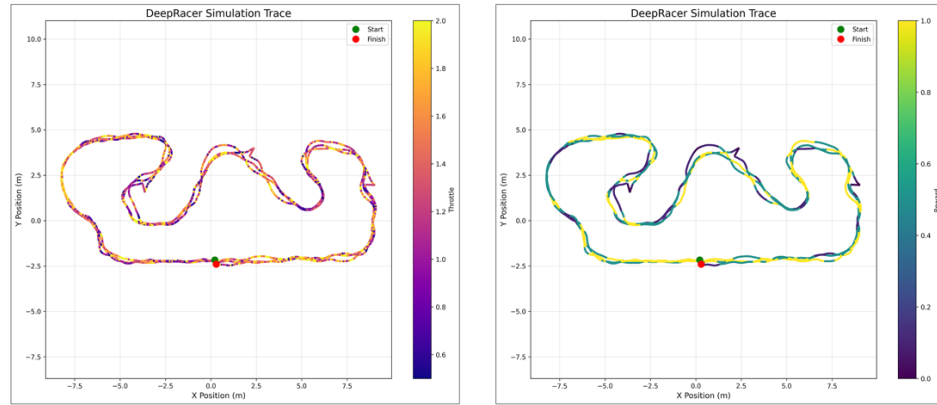


Fig 2: Throttle and Reward Graphs on Circuit de Barcelona-Catalunya

B. Speed Limit

In AWS DeepRacer, determining the optimal speed is crucial for balancing performance and stability. A higher speed allows the car to complete laps faster, but it also increases the risk of the vehicle veering off track, especially on sharp turns. Conversely, a lower speed offers greater control but may reduce overall efficiency. For this project, we experimented with various speed settings and found that an optimal range of 0.5 to 2.5 meters per second provided the best balance between lap time and track adherence. This range allowed the model to maintain sufficient momentum on straight sections while remaining stable around curves. The reward function was designed to encourage dynamic speed adjustment based on the car's heading and proximity to turns, helping improve both safety and performance.

C. Continuous Action Space –

The steering angle of the car determines how sharply the agent can turn. The steering angle parameters play a crucial role in the training of the model:

- Wider steering ranges increase the action space complexity, requiring more training time for the agent to learn optimal control.
- The steering angle limits determine how sharply your car can navigate turns - too narrow and it may not handle tight corners, too wide and it might make erratic movements.
- With continuous action spaces, the agent must train longer to learn to choose the optimal actions since it must learn to select precise values from the entire range. We trained our models for one hour as beyond that point, the model was overfitting, and the number of erratic movements and off-track count of the car were increasing.

The steering angle we chose for our model is in the range of 30 degrees to -30 degrees. This provides a perfect balance between driving stability and manoeuvrability. Increasing or decreasing the angle would result in unnecessary oscillations and zigzag movements causing the car to go off-track and resulting in a time penalty.

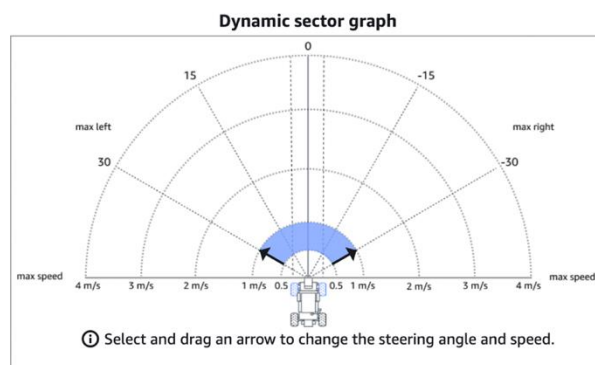


Fig 3: Dynamic Sector Graph

Hyperparameters – The hyperparameters used are the following:

- [1] **Gradient descent batch size (64)** - This batch size value balances learning quality with training speed - particularly important when dealing with continuous action spaces that require more exploration.

- [2] **Entropy (0.01)** - This low entropy value encourages more exploitation rather than exploration. Since continuous action spaces are already expansive, a lower entropy helps focus the agent on refining its learned steering and speed selections
- [3] **Discount Factor (0.99)** - The high discount factor means that the agent greatly values the future rewards. This high value is necessary for maintaining optimal racing lines and speeds through corners.
- [4] **Loss function (Huber)** - This loss function provides robustness against outliers. This is valuable for when the model is learning continuous steering and speed values and encounters extreme values. It helps the model to prevent being over influenced by these rare scenarios.
- [5] **Learning Rate (0.0003)** - The learning rate is essential when working with continuous action spaces where a small change in the steering angle can have a huge impact on performance. It allows for fine-grained adjustments to the model.

IV. CONTRIBUTIONS

Despite demonstrating strong performance on specific training circuits, our models exhibited a notable deficiency in their ability to generalise to previously unseen tracks. This issue was particularly evident during cross-circuit evaluations, where performance metrics showed significant degradation when models were applied outside their trained environments.

An example of this limitation was observed when evaluating one of our high-performing models. On Circuit de Barcelona - Catalunya, a scaled-down simulation of the real-world Formula 1 track, the model completed a lap in 00:43.808 without any off-track incidents. However, when tested on Jennens Super Speedway, the model's lap time increased drastically to 01:52.550, accompanied by 10 off-track deviations. These results suggest that the model had overfitted to the training circuit's geometry and dynamics and lacked the capacity to generalise to different track layouts and driving contexts.

This performance disparity showcased the need for generalised reward functions capable of guiding the learning process in a way that fosters adaptability across a wide range of track configurations. Our hypothesis was that more diverse training environments could lead to more robust driving policies.

To address this, we expanded the training dataset to include a selection of complex tracks featuring a variety of driving challenges, such as tight corners, hairpin turns, high-speed straights, and chicanes. The objective was to expose the model to a broader range of decision-making contexts during training, thereby improving its spatial reasoning, trajectory planning, and dynamic control capabilities.

The training process employed a sequential curriculum learning strategy, wherein each model was trained on one track at a time. Upon completion of training on a given track, the final model weights were cloned and used to initialise training on a new circuit. Importantly, the policy architecture, reward function, and hyperparameters were held constant throughout this process to ensure a controlled experimental setup. This approach allowed the model to retain beneficial learned behaviours while incrementally adapting to new environments.

To support this expanded training approach, we implemented reward functions designed to promote generalisable driving behaviours across diverse track environments. Rather than encouraging performance based on a single layout, the functions evaluated the car's actions using high-level driving principles that apply universally. They analyse upcoming sections of the track to estimate turn severity, allowing the model to adjust speed and position in anticipation of changes in direction. This enables the car to slow down for sharp curves, accelerate on straights, and follow an optimal racing line through corners. Additional rewards are given for maintaining alignment with the track direction and using smooth, appropriate steering.

V. RESULTS

To assess the impact of different reward functions on autonomous driving performance, we conducted a series of evaluations using four custom-designed reward functions - RFZ, RFB, RFY, and RFT across two AWS DeepRacer circuits: Jennens Super Speedway and Circuit de Barcelona-Catalunya. Each function was designed to promote different driving priorities, such as speed, stability, and smooth cornering. Performance was assessed using three metrics: raw lap time, number of off-track penalties (each incurring a 3-second penalty), and final adjusted lap time.

Table 2 presents the performance of each reward function on Jennens Super Speedway, a relatively simple and high-speed track. RFZ recorded the fastest raw lap time of 00:35.803, though it incurred one penalty, resulting in a final time of 00:38.803. While RFT completed the lap without penalties, it posted the slowest raw time at 00:47.043, indicating a more conservative driving behaviour. RFY, despite having a competitive raw time, suffered from two penalties, ultimately lowering its overall performance.

Jennens Super Speedway	Time	Penalty	Final Time
RFZ	00:35:803	1	00:38:803

RFB	00:37:132	1	00:40:132
RFY	00:38:050	2	00:44:050
RFT	00:47:043	0	00:47:043

Table 2: Jennens Super Speedway Lap Times

A different trend emerged on Circuit de Barcelona-Catalunya, a more complex track with tighter turns and varied curvature. As shown in **Table 3**, RFY achieved the best overall result with a final time of 00:42.206, balancing speed with minimal deviation. Conversely, RFZ, which had performed well on the simpler Jennens track, recorded the slowest final time of 00:58:591 due to a combination of slower pace and four off-track penalties. This highlights how reward functions tuned for high-speed straightaways may not transfer well to technical circuits requiring more nuanced control.

Circuit de Barcelona-Catalunya	Time	Penalty	Final Time
RFY	00:39:206	1	00:42:206
RFT	00:47:528	0	00:47:528
RFB	00:45:465	2	00:51:465
RFZ	00:46:591	4	00:58:591

Table 3: Circuit de Barcelona-Catalunya Lap Times

To test the generalisation ability of our trained models, as part of our curriculum learning approach, we implemented two training sequences: one starting with Jennens Super Speedway followed by Circuit de Barcelona-Catalunya, and the reverse. Both models were then evaluated on the 2022 re:Invent Championship track - an unseen and more complex environment. The reward function and other parameters for the models were kept constant across training sequences.

The model trained in the Jennens-first sequence exhibited superior generalisation performance, particularly on the unseen re:Invent track, where it completed a lap over 7 seconds faster than the reverse-trained model as shown in **Table 4**. This suggests that beginning training on simpler, more structured tracks promotes the development of broadly applicable driving policies, which can then be refined on more complex circuits.

Track	Jennens + Catalunya	Catalunya + Jennens
Jennens Super Speedway	00:46:525	01:08:203
Circuit de Barcelona-Catalunya	01:03:398	01:00:865
2022 re:Invent Championship	00:41:941	00:49:471

Table 4: Cross-Track Generalisation Performance by Training Order

VI. CONCLUSION

Our AWS DeepRacer project revealed that while models can achieve excellent performance on specific training tracks, generalization to unfamiliar environments remains a significant challenge. Through comparative analysis of PPO and SAC algorithms, optimization of speed parameters (0.5-2.5 m/s), and careful reward function design, we identified that single-track training leads to overfitting. The key finding was that a curriculum learning approach—starting with simpler tracks before progressing to complex circuits—substantially improved cross-track performance, with our Jennens-first sequence outperforming the reverse training order on unseen environments.

Our results demonstrate that successful autonomous racing requires balancing speed with control through appropriate reward functions tailored to track characteristics. This research establishes that effective autonomous racing agents depend not only on sophisticated algorithms but also on training methodologies that systematically build generalization capabilities across diverse driving conditions.